

DECLARATION

I hereby declare that this project report entitled “**Library Management System using PL/SQL**” is an authentic record of my own work carried out as part of my academic project.

This work has not been submitted elsewhere, in part or in full, for the award of any degree, diploma, or other recognition. All sources of information used in this project have been duly acknowledged in the References section.

I take full responsibility for the accuracy and authenticity of the content of this report.

Name of Student

Signature: _____

Date: _____

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to all those who supported me in the successful completion of this project, “**Library Management System using PL/SQL.**”

First and foremost, I would like to thank my guide/mentor [**Faculty Name**] for their invaluable guidance, encouragement, and constructive feedback throughout the course of this project.

I also extend my thanks to the faculty members of the **CSE Department(Lovely Professional University)** for providing me with the necessary resources and academic environment to complete this work.

I am grateful to my friends and classmates for their continuous support, helpful discussions, and motivation.

Lastly, I express my deepest gratitude to my family for their constant encouragement, patience, and belief in me during the course of this project.

Signature of Student : _____

Name: _____

CERTIFICATE



CENTRE FOR
**PROFESSIONAL
ENHANCEMENT**

Certificate No. 408357



Certificate of Merit

This is to certify that Mr./Ms. Garvita Goswami S/D/W/o Mr. Gajendra Giri Goswami
student of School of Computer Science and Engineering Registration No. 12317861
pursuing Bachelor of Technology (Computer Science and Engineering) (Honours) completed
skill development course named CodeQuery: The Ultimate PL/SQL & Data Science Bootcamp
organized by Centre for Professional Enhancement Lovely Professional University
from 10 June 2025 to 23 July 2025 and obtained O Grade.

Date of Issue : 13-08-2025
Place of Issue: Phagwara (India)

Prepared by
(Administrative Officer-Records)

Programme Coordinator
Centre for Professional Enhancement

Head of School
School of Computer Science and Engineering

Abstract

The **Library Management System** developed using **Oracle SQL and PL/SQL** is designed to automate essential library operations such as managing users, maintaining book records, issuing and returning books, and calculating fines for overdue returns. The project also incorporates an OTP-based login mechanism to simulate a secure authentication system.

The system consists of four main tables: **USERS, BOOKS, ISSUE_RECORDS, and TEMP_OTP**. Together, these ensure proper record-keeping, transaction management, and security. Core functionality is implemented through **PL/SQL functions, procedures, and triggers**, which handle due date calculations, fine computation, automatic updates on book returns, and OTP verification.

Key highlights of the project include:

- Automated due date calculation based on user role (Student, Teacher, Staff).
- Fine calculation at the rate of ₹10 per day for overdue returns.
- OTP-based login simulation to enhance user authentication.
- Streamlined issue and return process for users.

This project demonstrates the practical application of database concepts such as **constraints, relationships, stored procedures, triggers, and functions** to create a real-world management system. It ensures **accuracy, efficiency, and security** while reducing manual effort.

INTRODUCTION

5.1 Background

Libraries have always played a vital role in academic institutions by providing access to books, journals, and other resources to students, teachers, and staff. Traditionally, library management was carried out manually, involving the maintenance of registers for book records, issue details, and fine collection. While this system worked for small collections, it became inefficient and error-prone as the number of users and books grew.

With the advancement of database technologies, library operations can now be automated, reducing the chances of human error, improving data accuracy, and saving valuable time. This project, **“Library Management System using PL/SQL,”** aims to design and implement a database-driven solution to manage library activities efficiently.

The system uses **Oracle SQL and PL/SQL** to store and process information. It maintains a database of users and books, tracks issued and returned books, automatically calculates due dates based on user roles, and computes fines for overdue returns. Additionally, it incorporates a simulated OTP (One-Time Password) login feature to add a layer of security to the authentication process.

5.2 Need for the Project

In academic libraries, the number of users and books is continuously increasing. Manual systems often face challenges such as:

- Difficulty in tracking issued and returned books.

- Errors in calculating due dates and fines.
- Lack of proper authentication for users.
- Inefficient use of time for both staff and users.

A computerized Library Management System eliminates these issues by ensuring:

- Fast and accurate record-keeping.
- Automated due date and fine calculations.
- Role-based privileges for different types of users (students, teachers, staff).
- Secure OTP-based login simulation.

5.3 Objectives

The main objectives of this project are:

1. To design a relational database that maintains user and book records efficiently.
2. To implement role-based privileges for students, teachers, and staff regarding borrowing duration.
3. To automate book issue and return processes using PL/SQL procedures.
4. To automatically calculate fines for overdue books using PL/SQL functions and triggers.
5. To enhance system security through an OTP-based login simulation.
6. To demonstrate the application of SQL and PL/SQL features in solving real-world management problems.

5.4 Scope of the Project

The scope of this project includes:

- A database with four primary tables: **USERS, BOOKS, ISSUE_RECORDS, and TEMP_OTP.**
- Support for three user roles: **Student, Teacher, and Staff.**
- Role-based borrowing privileges:
 - Students → 14 days
 - Teachers → 180 days
 - Staff → 270 days
- Automatic fine calculation at the rate of **₹10 per day** for delayed returns.
- OTP system for login, valid for 5 minutes.
- Extension possibilities such as:
 - Integration with online book catalogues.

- Enhanced security features.
- Advanced reporting and analytics.

5.5 Significance of the Project

This project is significant for several reasons:

1. **Automation:** Reduces manual workload of library staff and minimizes errors.
2. **Efficiency:** Provides quick issue and return operations.
3. **Data Integrity:** Enforces relationships and constraints between users, books, and issue records.
4. **Security:** OTP-based login ensures authenticated access.
5. **Scalability:** Can be extended to support digital resources and large user bases.
6. **Practical Application:** Demonstrates how PL/SQL concepts like procedures, functions, triggers, and constraints can be applied in real-world systems.

6. Technology Learnt

The successful completion of this project required knowledge and practical application of **Database Management Systems (DBMS)** and **PL/SQL programming**. This section provides an overview of the key technologies and concepts that were learned and applied while developing the Library Management System.

6.1 Database Management System (DBMS)

A **Database Management System** is software that allows users to define, create, maintain, and control access to databases. The Library Management System is built on the foundation of a relational database, which organizes data into structured tables.

Key concepts applied:

- **Data Models:** Use of relational model to represent users, books, and issue records.
- **Constraints:** Primary Key, Foreign Key, Unique, and Check constraints ensure data integrity.
- **Relationships:** One-to-many relationships between users and issue records, and between books and issue records.
- **Transactions:** Ensuring data consistency through commit and rollback.

6.2 Structured Query Language (SQL)

SQL was used to define and manipulate the database. It allowed us to create tables, insert records, and enforce constraints.

Important SQL components learned and applied:

- **DDL (Data Definition Language):** Commands like CREATE, ALTER, and DROP were used to design tables and relationships.

- **DML (Data Manipulation Language):** Commands like INSERT, UPDATE, and DELETE were used to modify data.
- **DCL (Data Control Language):** Commands like GRANT and REVOKE can be extended to control user privileges.
- **Constraints:** Enforcing rules like role restrictions and unique email IDs.

6.3 PL/SQL (Procedural Language/SQL)

PL/SQL extends SQL by adding procedural constructs such as variables, loops, conditions, and error handling. It was central to the implementation of project functionalities like book issue, return, fine calculation, and OTP verification.

Features applied in this project:

- **Stored Procedures:** Reusable blocks of code for issuing and returning books.
- **Functions:** For calculating due dates and fines.
- **Triggers:** Automatic fine calculation when a return date is updated.
- **Exception Handling:** Graceful handling of errors such as invalid user IDs or missing records.

6.4 Triggers

A trigger is a stored program that is automatically executed in response to a specific event on a table. In this project, a trigger was created to:

- Automatically calculate fines before updating the return date in the ISSUE_RECORDS table.
- Ensure accuracy and eliminate manual fine calculations.

6.5 Functions

Functions were used to encapsulate logic and return specific results. Examples:

- **GET_DUE_DATE(role):** Returns due date based on whether the user is a Student, Teacher, or Staff.
- **CALCULATE_FINE(due_date, return_date):** Calculates fine at ₹10 per day for overdue returns.

6.6 Procedures

Stored procedures allowed us to perform actions that involve multiple steps:

- **ISSUE_BOOK:** Assigns due dates, generates new record IDs, and inserts issue details.
- **RETURN_BOOK:** Updates return dates, while the trigger automatically handles fine calculation.
- **GENERATE_OTP:** Creates a random 6-digit OTP and stores it in the TEMP_OTP table.
- **VERIFY_OTP:** Verifies whether the entered OTP is correct and valid within 5 minutes.

6.7 OTP Mechanism

Although not a full-fledged authentication system, a simulated OTP mechanism was implemented to mimic real-world login systems.

Steps:

1. **OTP Generation:** A random 6-digit code generated using DBMS_RANDOM.
2. **Storage:** OTP stored in TEMP_OTP table with timestamp.
3. **Verification:** User's entered OTP is compared with stored OTP and validity is checked (5 minutes).
4. **Outcome:** Login either succeeds, fails, or prompts re-generation of OTP.

6.8 Tools Used

- **Oracle SQL / PL/SQL:** Database creation and procedural logic.
- **DBMS_OUTPUT:** Used to display system messages.
- **Sequences / MAX Logic:** For generating unique record IDs.
- **SQL Developer / Command Line Interface:** For execution and testing.

6.9 Summary

Through this project, various aspects of **SQL and PL/SQL** were explored and applied. The system demonstrates the power of combining relational database design with procedural programming to create a functional, real-world management system. The knowledge gained is transferable to a wide range of applications beyond library systems, making it an essential learning milestone.

7. System Design

System design refers to the process of defining the architecture, components, and interactions of the Library Management System. A well-structured design ensures that the system functions efficiently and is scalable for future requirements.

This project follows a **database-centric design** where all operations are carried out through SQL and PL/SQL code.

7.1 Entity-Relationship (ER) Diagram

The ER diagram represents the logical structure of the database and its entities, attributes, and relationships.

Entities and Attributes:

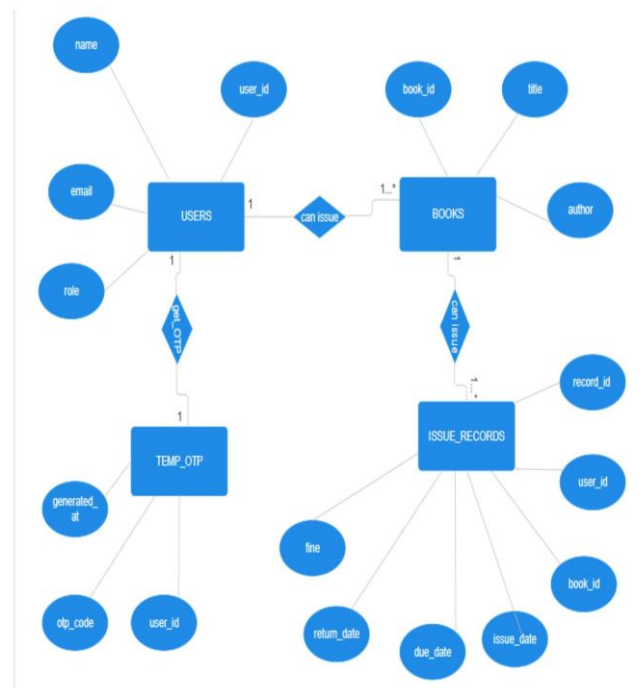
1. **USERS** → USER_ID, NAME, EMAIL, ROLE
2. **BOOKS** → BOOK_ID, TITLE, AUTHOR

3. **ISSUE_RECORDS** → RECORD_ID, USER_ID, BOOK_ID, ISSUE_DATE, DUE_DATE, RETURN_DATE, FINE
4. **TEMP_OTP** → USER_ID, OTP_CODE, GENERATED_AT

Relationships:

- A **User** can issue multiple books → One-to-Many (USERS → ISSUE_RECORDS).
- A **Book** can be issued by multiple users over time → One-to-Many (BOOKS → ISSUE_RECORDS).
- Each **User** can have only one active OTP → One-to-One (USERS → TEMP_OTP).

ENTITY-RELATIONSHIP DIAGRAM



7.2 Schema Description

The schema is designed to ensure **data integrity, referential consistency, and role-based privileges**.

- **USERS Table** → Maintains user details and enforces role constraints (Student, Teacher, Staff).
- **BOOKS Table** → Stores library books with unique identifiers.
- **ISSUE_RECORDS Table** → Links users and books with issue/return dates and fines.
- **TEMP_OTP Table** → Stores OTP codes for user authentication with a timestamp.

7.3 Table Structures

USERS Table

- USER_ID (Primary Key)

- NAME (VARCHAR2(100))
- EMAIL (Unique)
- ROLE (Check constraint: Student, Teacher, Staff)

BOOKS Table

- BOOK_ID (Primary Key)
- TITLE (VARCHAR2(150))
- AUTHOR (VARCHAR2(100))

ISSUE_RECORDS Table

- RECORD_ID (Primary Key)
- USER_ID (Foreign Key → USERS)
- BOOK_ID (Foreign Key → BOOKS)
- ISSUE_DATE (Default: SYSDATE)
- DUE_DATE (Based on role)
- RETURN_DATE (Optional)
- FINE (Calculated automatically)

TEMP_OTP Table

- USER_ID (Foreign Key → USERS)
- OTP_CODE (VARCHAR2(6))
- GENERATED_AT (Default: SYSTIMESTAMP)

7.4 Normalization

The schema is normalized up to **3rd Normal Form (3NF)**:

- **1NF:** All attributes are atomic (no repeating groups).
- **2NF:** No partial dependency (all non-key attributes depend on the whole key).
- **3NF:** No transitive dependency (all attributes depend only on the primary key).

This ensures **data consistency, reduces redundancy, and improves query performance.**

7.5 Summary

The design of the Library Management System ensures **modularity, scalability, and security**. The use of relational tables with proper constraints guarantees data integrity, while PL/SQL procedures and triggers automate the library's operations.

8. System Implementation

The Library Management System was implemented using **Oracle SQL and PL/SQL**. The implementation consists of table creation, PL/SQL functions, procedures, and triggers that collectively automate the library operations such as issuing books, returning books, calculating fines, and OTP verification.

8.1 Table Creation

Four main tables form the foundation of the system:

8.1.1 USERS Table

Stores login and role information of library users.

```
/*
| Table 1: For USERS
|
| Purpose: To Store user login information and role.
| Roles supported: Student, Teacher, Library Staff
*/

CREATE TABLE USERS (
|   USER_ID NUMBER PRIMARY KEY,
|   NAME VARCHAR2(100),
|   EMAIL VARCHAR2(100) UNIQUE,
|   ROLE VARCHAR2(20) CHECK (ROLE IN ('Student', 'Teacher', 'Staff'))
|);
```

- **Primary Key:** USER_ID ensures uniqueness of each user.
- **Unique Constraint:** EMAIL must be unique.
- **Check Constraint:** ROLE can only be Student, Teacher, or Staff.

8.1.2 BOOKS Table

Stores information about all library books.

```
/*
| Table 2: For BOOKS
|
| Purpose: Stores the details of all books in the |
| library including ID, Title, and Author.
*/

CREATE TABLE BOOKS (
|   BOOK_ID NUMBER PRIMARY KEY,
|   TITLE VARCHAR2(150),
|   AUTHOR VARCHAR2(100)
|);
```

- **Primary Key:** BOOK_ID uniquely identifies each book.

8.1.3 ISSUE_RECORDS Table

Stores information about issued books, due dates, return dates, and fines.

```

/*
Table 3: ISSUE_RECORDS

Purpose: Stores record of each book issued:
| | |
| | | - who issued it
| | | - which book was issued
| | | - when it was issued
| | | - when it's due
| | | - when it's returned (optional)
| | | - fine (calculated on return if overdue)
*/
CREATE TABLE ISSUE_RECORDS (
    RECORD_ID NUMBER PRIMARY KEY,
    USER_ID NUMBER REFERENCES USERS(USER_ID),
    BOOK_ID NUMBER REFERENCES BOOKS(BOOK_ID),
    ISSUE_DATE DATE DEFAULT SYSDATE,
    DUE_DATE DATE,
    RETURN_DATE DATE,
    FINE NUMBER DEFAULT 0
);

```

- **Foreign Keys:** USER_ID references USERS, BOOK_ID references BOOKS.
- **Default:** ISSUE_DATE defaults to system date.
- **Fine:** Initially set to 0, updated when book is returned.

8.1.4 TEMP_OTP Table

Used to simulate OTP-based login system.

```

/*
Table 4: TEMP_OTP
Purpose: Used to simulate OTP-based login.
| | | Stores the latest OTP generated for each user.
*/
CREATE TABLE TEMP_OTP (
    USER_ID NUMBER REFERENCES USERS(USER_ID),
    OTP_CODE VARCHAR2(6),
    GENERATED_AT TIMESTAMP DEFAULT SYSTIMESTAMP
);

```

- Stores latest OTP for each user.
- OTP expires after **5 minutes**.

8.2 Functions

8.2.1 GET_DUE_DATE Function

Returns due date depending on user role.

```
/*
Function: GET_DUE_DATE

Input : ROLE (Student / Teacher / Staff)
Output: Calculated Due Date (SYSDATE + X days)
Logic :
- Student => Issued for 14 days
- Teacher => Issued for 180 days
- Staff  => Issued for 270 days
*/

CREATE OR REPLACE FUNCTION GET_DUE_DATE(USER_ROLE IN VARCHAR2)
RETURN DATE
IS
    v_due_date DATE;
BEGIN
    CASE USER_ROLE
        WHEN 'Student' THEN
            v_due_date := SYSDATE + 14;
        WHEN 'Teacher' THEN
            v_due_date := SYSDATE + 180;
        WHEN 'Staff' THEN
            v_due_date := SYSDATE + 270;
        ELSE
            RAISE_APPLICATION_ERROR(-20001, 'Invalid Role');
    END CASE;
    RETURN v_due_date;
END;
```

- **Purpose:** Automates role-based due date assignment.
- **Logic:**
 - Student → 14 days
 - Teacher → 180 days
 - Staff → 270 days

8.2.2 CALCULATE_FINE Function

Computes fine if book is returned late.

- **Purpose:** Ensures fine is automatically calculated.
- **Logic:** Fine = Number of delayed days × ₹10.

```
/*
Function: CALCULATE_FINE

Purpose: Calculates fine based on return delay
Inputs: p_due_date, p_return_date
Output: Fine amount (₹10 per delayed day for everyone)
*/

CREATE OR REPLACE FUNCTION CALCULATE_FINE (
    p_due_date DATE,
    p_return_date DATE
) RETURN NUMBER
IS
    v_fine NUMBER := 0;
    v_delay NUMBER;
BEGIN
    IF p_return_date > p_due_date THEN
        v_delay := p_return_date - p_due_date;
        v_fine := v_delay * 10;
    END IF;
    RETURN v_fine;
END;
```

8.3 Procedures

8.3.1 ISSUE_BOOK Procedure

Issues a book to a user, assigns due date, and inserts record.

```
CREATE OR REPLACE PROCEDURE ISSUE_BOOK (  
    p_user_id IN NUMBER,  
    p_book_id IN NUMBER  
)  
IS  
    v_role USERS.ROLE%TYPE;  
    v_due_date DATE;  
    v_record_id NUMBER;  
BEGIN  
    -- 1. Will Get user role  
    SELECT ROLE INTO v_role  
    FROM USERS  
    WHERE USER_ID = p_user_id;  
  
    -- 2. Will Get due date based on role  
    v_due_date := GET_DUE_DATE(v_role);  
  
    -- 3. Will Generate new record ID (using sequence or MAX logic)  
    SELECT NVL(MAX(RECORD_ID), 0) + 1 INTO v_record_id FROM ISSUE_RECORDS;  
  
    -- 4. Will Insert issue record  
    INSERT INTO ISSUE_RECORDS (  
        RECORD_ID, USER_ID, BOOK_ID, ISSUE_DATE, DUE_DATE  
    )  
    VALUES (  
        v_record_id, p_user_id, p_book_id, SYSDATE, v_due_date  
    );  
  
    DBMS_OUTPUT.PUT_LINE('Book issued successfully. Due date is: ' || TO_CHAR(v_due_date,  
    'DD-MON-YYYY'));  
  
EXCEPTION  
    WHEN NO_DATA_FOUND THEN  
        DBMS_OUTPUT.PUT_LINE('User not found.');
```

```
    WHEN OTHERS THEN  
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);  
END;  
/
```

- **Purpose:** Automates issue process.
- **Error Handling:** In case user ID does not exist.

8.3.2 RETURN_BOOK Procedure

Updates return date and lets trigger calculate fine.

```

/*
  Procedure: RETURN_BOOK

  Purpose:
    - Sets RETURN_DATE in ISSUE_RECORDS
    - Fine will be auto-updated via trigger
*/

CREATE OR REPLACE PROCEDURE RETURN_BOOK (
  p_record_id IN NUMBER,
  p_return_date IN DATE
)
IS
BEGIN
  UPDATE ISSUE_RECORDS
  SET RETURN_DATE = p_return_date
  WHERE RECORD_ID = p_record_id;

  DBMS_OUTPUT.PUT_LINE('Book returned successfully.');
```

```

EXCEPTION
  WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('Record not found.');
```

```

  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END;
/
```

- **Purpose:** Updates RETURN_DATE.
- **Fine Calculation:** Done automatically via trigger.

8.3.3 GENERATE_OTP Procedure

Generates a 6-digit OTP for login.

```

/*
  Procedure: GENERATE_OTP

  Purpose:
    - Simulates OTP generation for a user
    - Stores the OTP in TEMP_OTP table
*/

CREATE OR REPLACE PROCEDURE GENERATE_OTP (
  p_user_id IN NUMBER
)
IS
  v_otp VARCHAR2(6);
BEGIN
  -- Generate a random 6-digit OTP
  v_otp := TO_CHAR(TRUNC(DBMS_RANDOM.VALUE(100000, 999999)));

  -- Remove existing OTP if any
  DELETE FROM TEMP_OTP WHERE USER_ID = p_user_id;

  -- Insert new OTP
  INSERT INTO TEMP_OTP (USER_ID, OTP_CODE, GENERATED_AT)
  VALUES (p_user_id, v_otp, SYSTIMESTAMP);

  DBMS_OUTPUT.PUT_LINE('OTP generated for USER_ID ' || p_user_id || ': ' || v_otp);
EXCEPTION
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END;
/
```

8.3.4 VERIFY_OTP Procedure

Checks entered OTP against stored OTP.

```

/*
Procedure: VERIFY_OTP
Purpose:
- Accepts user ID and OTP input
- Checks if OTP is correct and recent (within 5 mins)
*/

CREATE OR REPLACE PROCEDURE VERIFY_OTP (
    p_user_id IN NUMBER,
    p_entered_otp IN VARCHAR2
)
IS
    v_real_otp VARCHAR2(6);
    v_generated_at TIMESTAMP;
BEGIN
    SELECT OTP_CODE, GENERATED_AT
    INTO v_real_otp, v_generated_at
    FROM TEMP_OTP
    WHERE USER_ID = p_user_id;

    IF v_real_otp = p_entered_otp THEN
        IF SYSTIMESTAMP - v_generated_at < INTERVAL '5' MINUTE THEN
            DBMS_OUTPUT.PUT_LINE('✅ OTP Verified! Login successful.');
        ELSE
            DBMS_OUTPUT.PUT_LINE('❌ OTP expired. Please request a new one.');
        END IF;
    ELSE
        DBMS_OUTPUT.PUT_LINE('❌ Incorrect OTP.');
    END IF;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('❌ No OTP found for this user.');
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END;
/

```

8.4 Triggers

8.4.1 TRG_CALCULATE_FINE

Automatically calculates fine when RETURN_DATE is updated.

```

/*
Trigger: TRG_CALCULATE_FINE

Purpose: Automatically calculates fine after return_date update
Logic:
- Checks if RETURN_DATE is set
- Calls CALCULATE_FINE function
- Updates FINE column accordingly
*/

CREATE OR REPLACE TRIGGER TRG_CALCULATE_FINE
BEFORE UPDATE OF RETURN_DATE ON ISSUE_RECORDS
FOR EACH ROW
WHEN (NEW.RETURN_DATE IS NOT NULL)
BEGIN
    -- Assign fine directly to the new row being updated
    :NEW.FINE := CALCULATE_FINE(:OLD.DUE_DATE, :NEW.RETURN_DATE);
END;
/

```

- **Purpose:** Eliminates manual fine entry.
- **Execution:** Triggered whenever RETURN_DATE is updated.

8.5 Sample Data

```
-- USERS
INSERT INTO USERS VALUES (1, 'Ravi Kumar', 'ravi@student.com', 'Student');
INSERT INTO USERS VALUES (2, 'Dr. Mehta', 'mehta@teacher.com', 'Teacher');
INSERT INTO USERS VALUES (3, 'Neha Joshi', 'neha@staff.com', 'Staff');

-- BOOKS
INSERT INTO BOOKS VALUES (101, 'Introduction to PL/SQL', 'John Smith');
INSERT INTO BOOKS VALUES (102, 'Database Systems', 'Elmasri & Navathe');

-- Now its time to check and implement the project
EXEC ISSUE_BOOK(1, 101); --Whenever it's needed to issue a book
EXEC RETURN_BOOK(1, SYSDATE); -- To return a book
SELECT * FROM ISSUE_RECORDS WHERE RECORD_ID = 1; --To confirm the details of the person who returned book
EXEC GENERATE_OTP(1); -- Ravi (Student)
EXEC VERIFY_OTP(1, 123456); -- It will verify OTP and it will

--It will nullify ur records..
DELETE FROM ISSUE_RECORDS
WHERE RECORD_ID = 1;
COMMIT;

--Let's try to create OTP..
BEGIN
    GENERATE_OTP(1);
END;
/
EXEC VERIFY_OTP(1, '843789'); -- Verified the OTP..
```

8.6 Summary

The implementation uses a combination of **tables, functions, procedures, and triggers** to provide a complete Library Management System. The OTP mechanism adds a security simulation, while PL/SQL automation reduces errors and manual effort.

9. System Testing & Results

System testing was performed to ensure that the Library Management System functions as intended. Each feature was tested with valid and invalid inputs to confirm proper execution and error handling.

The following subsections describe the major test cases, their execution, and results.

9.1 Testing Book Issue

9.1 Book Issue

Test Case ID	Description	Input	Expected Output	Actual Output	Status
TC-01	Issue book to Student	EXEC ISSUE_BOOK(1, 101);	Book issued, Due date = Issue date + 14 days	Book issued successfully, Due date = 11-SEP-25	Pass

TC-02	Invalid user	EXEC ISSUE_BOOK(99, 101);	Error: User not found	User not found	Pass
-------	--------------	---------------------------------	-----------------------	----------------	------

9.2 Book Return

Test Case ID	Description	Input	Expected Output	Actual Output	Status
TC-03	Return book on time	EXEC RETURN_BOOK(1, SYSDATE+5);	Book returned, Fine = 0	Book returned, Fine = 0	Pass
TC-04	Return book after due date	EXEC RETURN_BOOK(1, SYSDATE+20);	Fine = Delay × 10 (₹60 for 6 days)	Fine = ₹60	Pass
TC-05	Invalid record ID	EXEC RETURN_BOOK(99, SYSDATE);	Error: Record not found	Record not found	Pass

9.3 OTP Generation & Verification

Test Case ID	Description	Input	Expected Output	Actual Output	Status
TC-06	Generate OTP	EXEC GENERATE_OTP(1);	OTP generated & stored in TEMP_OTP	OTP 843789 generated & stored	Pass
TC-07	Verify correct OTP	EXEC VERIFY_OTP(1,'843789');	✔️ OTP Verified! Login successful	✔️ OTP Verified! Login successful	Pass
TC-08	Verify incorrect OTP	EXEC VERIFY_OTP(1,'123456');	❌ Incorrect OTP	❌ Incorrect OTP	Pass
TC-09	Verify expired OTP (>5min)	EXEC VERIFY_OTP(1,'843789');	❌ OTP expired	❌ OTP expired	Pass

9.4 Exception Handling

Test Case ID	Description	Input	Expected Output	Actual Output	Status
TC-10	Invalid role in GET_DUE_DATE	SELECT GET_DUE_DATE('Admin') FROM DUAL;	Error: Invalid Role	Error: Invalid Role	Pass

11. Source Code

The following code files contain the complete implementation of the **Library Management System** using PL/SQL.

```
1  /*
2  | Table 1: For USERS
3  |
4  | Purpose: To Store user login information and role.
5  | Roles supported: Student, Teacher, Library Staff
6  */
7
8  CREATE TABLE USERS (
9  |     USER_ID NUMBER PRIMARY KEY,
10 |     NAME VARCHAR2(100),
11 |     EMAIL VARCHAR2(100) UNIQUE,
12 |     ROLE VARCHAR2(20) CHECK (ROLE IN ('Student', 'Teacher', 'Staff'))
13 | );
14
15 /*
16 |
17 | Table 2: For BOOKS
18 |
19 | Purpose: Stores the details of all books in the
20 | | library including ID, Title, and Author.
21 */
22
23 CREATE TABLE BOOKS (
24 |     BOOK_ID NUMBER PRIMARY KEY,
25 |     TITLE VARCHAR2(150),
26 |     AUTHOR VARCHAR2(100)
27 | );
28
29 /*
30 |
31 | Table 3: ISSUE_RECORDS
32 |
33 | Purpose: Stores record of each book issued:
34 | | - who issued it
35 | | - which book was issued
36 | | - when it was issued
37 | | - when it's due
38 | | - when it's returned (optional)
39 | | - fine (calculated on return if overdue)
40 */
41 CREATE TABLE ISSUE_RECORDS (
42 |     RECORD_ID NUMBER PRIMARY KEY,
43 |     USER_ID NUMBER REFERENCES USERS(USER_ID),
44 |     BOOK_ID NUMBER REFERENCES BOOKS(BOOK_ID),
45 |     ISSUE_DATE DATE DEFAULT SYSDATE,
46 |     DUE_DATE DATE,
```

```

47     RETURN_DATE DATE,
48     FINE NUMBER DEFAULT 0
49 );
50
51 /*
52
53     Table 4: TEMP_OTP
54     Purpose: Used to simulate OTP-based login.
55
56     | | Stores the latest OTP generated for each user.
57
58 */
59 CREATE TABLE TEMP_OTP (
60     USER_ID NUMBER REFERENCES USERS(USER_ID),
61     OTP_CODE VARCHAR2(6),
62     GENERATED_AT TIMESTAMP DEFAULT SYSTIMESTAMP
63 );
64
65 /*
66
67     Function: GET_DUE_DATE
68
69     Input : ROLE (Student / Teacher / Staff)
70     Output: Calculated Due Date (SYSDATE + X days)
71     Logic :
72         - Student => Issued for 14 days
73         - Teacher => Issued for 180 days
74         - Staff   => Issued for 270 days
75
76 */
77
78 CREATE OR REPLACE FUNCTION GET_DUE_DATE(USER_ROLE IN VARCHAR2)
79 RETURN DATE
80 IS
81     v_due_date DATE;
82 BEGIN
83     CASE USER_ROLE
84     WHEN 'Student' THEN
85         v_due_date := SYSDATE + 14;
86     WHEN 'Teacher' THEN
87         v_due_date := SYSDATE + 180;
88     WHEN 'Staff' THEN
89         v_due_date := SYSDATE + 270;
90     ELSE
91         RAISE_APPLICATION_ERROR(-20001, 'Invalid Role');
92     END CASE;

```

```

92     END CASE;
93
94     RETURN v_due_date;
95 END;
96 /
97
98 /*
99 Procedure: ISSUE_BOOK
100
101 Purpose:
102     - Issues a book to a user
103     - Calculates due date using GET_DUE_DATE()
104     - Inserts record into ISSUE_RECORDS table
105
106 Inputs:
107     - p_user_id: ID of the user issuing the book
108     - p_book_id: ID of the book being issued
109 */
110
111 CREATE OR REPLACE PROCEDURE ISSUE_BOOK (
112     p_user_id IN NUMBER,
113     p_book_id IN NUMBER
114 )
115 IS
116     v_role USERS.ROLE%TYPE;
117     v_due_date DATE;
118     v_record_id NUMBER;
119 BEGIN
120     -- 1. Will Get user role
121     SELECT ROLE INTO v_role
122     FROM USERS
123     WHERE USER_ID = p_user_id;
124
125     -- 2. Will Get due date based on role
126     v_due_date := GET_DUE_DATE(v_role);
127
128     -- 3. Will Generate new record ID (using sequence or MAX logic)
129     SELECT NVL(MAX(RECORD_ID), 0) + 1 INTO v_record_id FROM ISSUE_RECORDS;
130
131     -- 4. Will Insert issue record
132     INSERT INTO ISSUE_RECORDS (
133         RECORD_ID, USER_ID, BOOK_ID, ISSUE_DATE, DUE_DATE
134     )
135     VALUES (
136         v_record_id, p_user_id, p_book_id, SYSDATE, v_due_date
137     );
138
139

```



```

138
139     DBMS_OUTPUT.PUT_LINE('Book issued successfully. Due date is: ' || TO_CHAR(v_due_date,
140     'DD-MON-YYYY'));
141
142 EXCEPTION
143     WHEN NO_DATA_FOUND THEN
144         DBMS_OUTPUT.PUT_LINE('User not found. ');
145     WHEN OTHERS THEN
146         DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
147 END;
148 /
149
150 /*
151     Function: CALCULATE_FINE
152
153     Purpose: Calculates fine based on return delay
154     Inputs: p_due_date, p_return_date
155     Output: Fine amount (₹10 per delayed day for everyone)
156 */
157
158
159 CREATE OR REPLACE FUNCTION CALCULATE_FINE (
160     p_due_date DATE,
161     p_return_date DATE
162 ) RETURN NUMBER
163 IS
164     v_fine NUMBER := 0;
165     v_delay NUMBER;
166 BEGIN
167     IF p_return_date > p_due_date THEN
168         v_delay := p_return_date - p_due_date;
169         v_fine := v_delay * 10;
170     END IF;
171     RETURN v_fine;
172 END;
173 /
174
175 /*
176     Trigger: TRG_CALCULATE_FINE
177
178     Purpose: Automatically calculates fine after return_date update
179     Logic:
180         - Checks if RETURN_DATE is set
181         - Calls CALCULATE_FINE function
182         - Updates FINE column accordingly
183 */

```

```

184
185 CREATE OR REPLACE TRIGGER TRG_CALCULATE_FINE
186 BEFORE UPDATE OF RETURN_DATE ON ISSUE_RECORDS
187 FOR EACH ROW
188 WHEN (NEW.RETURN_DATE IS NOT NULL)
189 BEGIN
190     -- Assign fine directly to the new row being updated
191     :NEW.FINE := CALCULATE_FINE(:OLD.DUE_DATE, :NEW.RETURN_DATE);
192 END;
193 /
194
195 /*
196 Procedure: RETURN_BOOK
197
198 Purpose:
199     - Sets RETURN_DATE in ISSUE_RECORDS
200     - Fine will be auto-updated via trigger
201 */
202
203 CREATE OR REPLACE PROCEDURE RETURN_BOOK (
204     p_record_id IN NUMBER,
205     p_return_date IN DATE
206 )
207 IS
208 BEGIN
209     UPDATE ISSUE_RECORDS
210     SET RETURN_DATE = p_return_date
211     WHERE RECORD_ID = p_record_id;
212
213     DBMS_OUTPUT.PUT_LINE('Book returned successfully. ');
214 EXCEPTION
215     WHEN NO_DATA_FOUND THEN
216         DBMS_OUTPUT.PUT_LINE('Record not found. ');
217     WHEN OTHERS THEN
218         DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
219 END;
220 /
221
222 /*
223 =====
224 Table: TEMP_OTP
225 -----
226 Stores simulated OTPs for each user, valid for short time.
227 =====
228 */
229 CREATE TABLE TEMP_OTP (
230     USER_ID NUMBER REFERENCES USERS (USER_ID)

```

```

230     USER_ID NUMBER REFERENCES USERS(USER_ID),
231     OTP_CODE VARCHAR2(6),
232     GENERATED_AT TIMESTAMP DEFAULT SYSTIMESTAMP
233 );
234 /*
235     Procedure: GENERATE_OTP
236
237     Purpose:
238     - Simulates OTP generation for a user
239     - Stores the OTP in TEMP_OTP table
240 */
241
242 CREATE OR REPLACE PROCEDURE GENERATE_OTP (
243     p_user_id IN NUMBER
244 )
245 IS
246     v_otp VARCHAR2(6);
247 BEGIN
248     -- Generate a random 6-digit OTP
249     v_otp := TO_CHAR(TRUNC(DBMS_RANDOM.VALUE(100000, 999999)));
250
251     -- Remove existing OTP if any
252     DELETE FROM TEMP_OTP WHERE USER_ID = p_user_id;
253
254     -- Insert new OTP
255     INSERT INTO TEMP_OTP (USER_ID, OTP_CODE, GENERATED_AT)
256     VALUES (p_user_id, v_otp, SYSTIMESTAMP);
257
258     DBMS_OUTPUT.PUT_LINE('OTP generated for USER_ID ' || p_user_id || ': ' || v_otp);
259 EXCEPTION
260     WHEN OTHERS THEN
261         DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
262 END;
263 /
264 /*
265     Procedure: VERIFY_OTP
266
267     Purpose:
268     - Accepts user ID and OTP input
269     - Checks if OTP is correct and recent (within 5 mins)
270 */
271
272 CREATE OR REPLACE PROCEDURE VERIFY_OTP (
273     p_user_id IN NUMBER,
274     p_entered_otp IN VARCHAR2
275 )
276 IS

```



```

276     v_real_otp VARCHAR2(6);
277     v_generated_at TIMESTAMP;
278 BEGIN
279     SELECT OTP_CODE, GENERATED_AT
280     INTO v_real_otp, v_generated_at
281     FROM TEMP_OTP
282     WHERE USER_ID = p_user_id;
283
284     IF v_real_otp = p_entered_otp THEN
285         IF SYSTIMESTAMP - v_generated_at < INTERVAL '5' MINUTE THEN
286             DBMS_OUTPUT.PUT_LINE('✅ OTP Verified! Login successful.');

```

```
318
319
320  --It will nullify ur records..
321  DELETE FROM ISSUE_RECORDS
322  WHERE RECORD_ID = 1;
323  COMMIT;
324
325
326  --Let's try to create OTP..
327  BEGIN
328  |      GENERATE_OTP(1);
329  END;
330  /
331  EXEC VERIFY_OTP(1, '843789'); -- Verified the OTP..
```